Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Bir algoritmanın çalışma süresini ölçmenin yolları ve sorunları
- Çalışma süresini etkileyen sisteme bağlı ve sistemden bağımsız etkenler
- İşlem sayma: her işlemin kaç kez yapıldığını bulma (operation / frequency tablosu)
- Büyüme oranı (order of growth) ve asimptotik analiz (asymptotic analysis)
- Asimptotik gösterimler: Big-O, Big-Theta, Big-Omega
- Algoritma sınıfları: sabit, logaritmik, doğrusal, $n \log n$, karesel, üstel zaman
- Temel işlem (basic operation): en çok zaman alan işleme odaklanma
- Sırasız dizide sıralı arama (sequential search): en kötü, en iyi ve ortalama durum
- İkili arama (binary search): akış diyagramı, durma koşulu ve karmaşıklık hesabı
- Geriye doğru yerine koyma (backward substitution) ile $O(\log_2 n)$ sonucunun çıkarılması
- Sınav tipi soru: 0 ve 1'lerden oluşan sıralı dizide geçiş noktasını $O(\log n)$ zamanda bulma

Hocanın Özellikle Vurguladığı Kısımlar

- Algoritmanın önce doğru olması gerekir. Doğru çalıştıktan sonra önemli olan, zamanı ve yeri iyi kullanmasıdır.
- Çalışma süresini saatle ölçmek, iki algoritmayı karşılaştırmak için objektif bir yöntem değildir. Donanım, dil, derleyici ve işletim sistemi sonucu değiştirir.
- Karmaşıklık, girişteki eleman sayısı n'e bağlı olarak söylenir.
- Satır satır işlem saymak büyük algoritmalarda zahmetlidir ve gereksizdir. En çok zaman alan işi (genelde döngüler) bulup ona odaklanın.
- n sonsuza giderken katsayılar, sabitler ve küçük terimler önemsizdir. Yalnızca en anlamlı terim alınır.
- $m \cdot n$ gibi ifadelerde m'in durumunu gözden kaçırmayın. m de n gibi büyüyebiliyorsa karmaşıklık $O(m \cdot n)$ ya da $O(n^2)$ olur.
- Bu derste ve algoritma analizi dersinde ağırlık en kötü durumdadır (worst case).
- Ortalama durum (average case), sonuçların ortalamasını almak demek değildir. Rastgele bir giriş için beklenen adım sayısıdır.
- İkili aramada `left = mid` veya `right = mid` yazmak sık yapılan bir hatadır. Doğrusu `right = mid - 1` ve `left = mid + 1` olmalıdır.
- İkili aramada durma koşulu `left <= right` olmalıdır. Eşitlik durumunu atlamayın, aranan eleman tam bu durumda bulunabilir.
- Tasarımı kodla değil, akış diyagramıyla yapın. Hatayı çok daha çabuk bulursunuz.
- 0/1 dizisi sorusu hocanın sınavlarda sorduğu bir sorudur. Ders sonrası kendiniz çözmeniz önerildi.

Kısa Tekrar Notları

- Sisteme bağlı etkenler: donanım (işlemci, bellek, önbellek), yazılım (dil, derleyici ya da yorumlayıcı, derleyici optimizasyonu), işletim sistemi ve o an çalışan diğer uygulamalar.
- Sistemden bağımsız etkenler: algoritmanın kendisi ve giriş (eleman sayısı ve girişin durumu, örneğin dizinin sıralı ya da yarı sıralı olması).
- Büyüme oranı kuralları:
  - Katsayıları at: $4n \to n$
  - Sabitleri at: $n + 6 \to n$

  - En anlamlı terimi al: $3n^2 + 2n + 5 \to n^2$
- Örnekler: $n + \log n \to n$, $n^2 + n \to n^2$, $n \log n + n \to n \log n$.
- Asimptotik gösterimler:
  - Big-O: üst sınır (upper bound)
  - Big-Theta: sıkı sınır (tight bound)
  - Big-Omega: alt sınır (lower bound)
- Algoritma sınıfları:
  - Sabit zaman (constant time): $O(1)$
  - Logaritmik zaman (logarithmic time): $O(\log_2 n)$
  - Doğrusal zaman (linear time): $O(n)$
  - $n \log n$ zaman (linearithmic time): $O(n \log_2 n)$
  - Karesel zaman (quadratic time): $O(n^2)$
  - Üstel zaman (exponential time): $O(2^n)$
- Sıralı arama: en kötü durum $O(n)$, en iyi durum $O(1)$, ortalama durum $O(n)$.
- İkili arama: en kötü durum $O(\log_2 n)$. 1024 (2^{10}) elemanlı dizide en fazla 10 karşılaştırma yapılır.
- Toplam formülleri: $\sum_{i=1}^{n} 1 = n$ ve $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \sim n^2$.

Detaylı Açıklamalar

Çalışma Süresini Ölçmek

- İlk akla gelen yol, iki algoritmanın çalışma süresini ölçmektir.
- Ölçüm yaparken zamanlayıcı (timer), algoritmanın başladığı yerden hemen önce başlatılır ve bittiği yerde durdurulur.
- Giriş/çıkış gibi algoritmayla ilgisi olmayan işler ölçümün dışında tutulmalıdır.
- Derin öğrenme gibi çok karmaşık sistemlerde karmaşıklık hesabı zordur. Bu durumda "şu donanımda şu kadar sürdü" diye süre vermek çaresiz kalınca başvurulan bir yoldur.
- Temel algoritmalarda ise bu yöntem doğru değildir. Çünkü sonucu şu etkenler değiştirir:
  - Donanım: işlemci, bellek, verinin önbellekte (cache) olup olmaması
  - Yazılım: aynı program C, Java veya MATLAB'de çok farklı sürelerde çalışır. Derleyici (compiler) ya da yorumlayıcı (interpreter) kullanılması da fark yaratır.
  - Derleyici optimizasyonu: bazı derleyiciler kodu optimize eder. Bu seçenek açılıp kapatılabilir.
  - İşletim sistemi, ağ ve o an çalışan diğer uygulamalar
- Sistemden bağımsız iki etken vardır:
  - Algoritma: verimli bir algoritma daha kısa sürede çalışır.
  - Giriş: eleman sayısı ($n = 10$ ile $n = 100$ farklıdır) ve girişin durumu. Örneğin dizinin sıralı, yarı sıralı veya sırasız verilmesi süreyi değiştirir.
- Objektif bir ölçüm için çalışma zamanı, girişteki eleman sayısı n'e bağlı olarak ifade edilir.

İşlem Sayma Örneği

Hoca, dizideki sıfırları sayan şu kod parçası için her işlemin kaç kez yapıldığını tabloya döktü (ekrandan, 00:20:40):

```
int count = 0;
int i, n;
for (i = 0; i < n; i++)
{
    if (a[i] == 0)
        count++;
}
```

İşlem (operation)	Sayı (frequency)
Değişken tanımı (variable declaration)	3
Atama (assignment statement)	2
Küçüktür karşılaştırması (less than compare)	$n + 1$

İşlem (operation)	Sayı (frequency)
Eşittir karşılaştırması (equal to compare)	n
Dizi erişimi (array access)	n
Artırma (increment)	n ile $2n$ arası
Toplam	$4n + 6$ ile $5n + 6$ arası

- `i < n` karşılaştırması $n + 1$ kez yapılır. n kez döngüye girilir, $(n + 1)$'inci kontrolde $i = n$ olur ve döngüden çıkılır.
- Artırma sayısı:
  - `i++` her zaman n kez yapılır.
  - `count++` ise `a[i]` hiç sıfır değilse 0 kez, hep sıfırsa n kez yapılır.
  - Bu yüzden artırma sayısı en az n, en çok $2n$'dir.
- Bu sayımda atamanın, karşılaştırmanın kaç saat çevrimi (cycle) sürdüğüne bakılmaz. Yalnızca kaç kez yapıldığına bakılır.
- Hoca sordu: biri $5n + 6$, diğeri $5n + 9$ bulduysa hangi algoritma daha iyidir? Cevap: aralarında anlamlı bir fark yoktur. Bir atamanın fazla ya da eksik olması önemli değildir.

Büyüme Oranı ve Asimptotik Analiz

- Önemli olan, n çok büyüdüğünde (sonsuza giderken) ne olduğudur. Buna büyüme oranı (order of growth) denir.
- Büyüme oranı bulunurken:
  - Katsayılar atılır. $4n$ ile n arasında fark yoktur.
  - Sabitler atılır. 1 milyon ile 1 milyon artı 6 arasında fark yoktur.
  - En anlamlı terim alınır. n sonsuza giderken n, n^2'nin yanında etkisizdir.
- Karmaşıklığın makineden, derleyiciden ve dilden bağımsız olarak hesaplanmasına asimptotik analiz (asymptotic analysis) denir.
- Bu gösterim için kullanılan notasyona asimptotik gösterim (asymptotic notation) denir:
  - Big-O $O()$: üst sınır. Algoritmanın olabileceği en kötü durumu gösterir.
  - Big-Theta $\Theta()$: sıkı sınır. Algoritmanın kesin olarak ne olduğunu gösterir.
  - Big-Omega $\Omega()$: alt sınır. Algoritmanın en az ne olabileceğini gösterir. Örneğin seçmeli sıralama (selection sort) hiçbir girişte n^2'nin altına inemez.
- Eklemeli sıralama (insertion sort) için alt sınır n (en iyi durum), üst sınır n^2'dir.
- Bu derste yalnızca Big-O kullanılacak. Örneğin $4n^2 + 5n$ için $O(n^2)$ denir. Diğer gösterimler algoritma analizi dersinde ayrıntılı görülecek.

Ekranda verilen karşılaştırma ($A = 100n + 1$, $B = n^2 + n + 1$), ekrandan 00:35:30:

n	$A = 100n + 1$	$B = n^2 + n + 1$
10	1001	111
100	10001	10101
10000	1000001	100010001 (bkz. ders sonrası notlar)

- Küçük n için A daha büyüktür. n büyüdükçe n^2 terimi baskın hâle gelir ve B çok daha büyük olur.

Dikkat Edilmesi Gereken Durumlar

- $m \cdot n$ **karmaşıklığı:** m ayrı bir değişkendir.
  - Örneğin 10 000 öğrenci ve her öğrencinin 10 dersi varsa m'in önemi azdır.
  - m de n gibi büyüyebiliyorsa bu gözden kaçırılmamalıdır. Gösterim genelde $O(m \cdot n)$, nadiren $O(n^2)$ olur.
- $2n^2 + cn$ **karmaşıklığı:** sonuç $O(n^2)$'dir.
  - c bir sabittir ama dışarıdan gelen bir bilgiye göre belirleniyor olabilir.
  - c çok büyükse (örneğin 10^6) ve n küçükse (örneğin 1000), gerçek çalışmada cn terimi etkili olabilir. Bunun farkında olmak gerekir.

- Derste gelen soru: c'yi n kabul edersek ne olur? Cevap: cn terimi n^2 olur. Toplam yine $O(n^2)$ kalır, karmaşıklık değişmez.

Algoritma Sınıfları

- **Sabit zaman, $O(1)$:** n'e bağlı olmayan algoritmalar. Örnek: üçgenin eşkenar, ikizkenar veya çeşitkenar olduğunu bulmak; üç sayının en küçüğünü bulmak. 100 adım da sürse, 200 adım da sürse bu sayı değişmediği için karmaşıklık $O(1)$'dir.
- **Logaritmik zaman, $O(\log_2 n)$:** her adımda problemi ikiye bölen algoritmalar. Örnek: `while (n > 1) n = n / 2;` döngüsü ve ikili arama. 8 elemandan 4'e, 2'ye, 1'e inilir.
- **Doğrusal zaman, $O(n)$:** tek döngü. Örnek: dizinin en büyük elemanını bulmak.
- **$n \log n$ zaman, $O(n \log_2 n)$:** hızlı sıralama (quick sort), yığın sıralaması (heap sort), birleştirmeli sıralama (merge sort). Dönem sonunda görülecek.
- **Karesel zaman, $O(n^2)$:** iç içe iki döngü (double loop). Örnek: seçmeli sıralama, kabarcık sıralaması (bubble sort), matris üzerinde iki boyutlu işlemler.
- **Üstel zaman, $O(2^n)$:** olası bütün durumların değerlendirildiği algoritmalar. Büyük n için makul sürede çözülemezler.

Temel İşlem (Basic Operation)

- Karmaşıklık hesabında atama, karşılaştırma, `return` gibi işlemler tek tek sayılmaz. Zaten sonunda katsayılar ve sabitler atılacaktır.
- Algoritmadaki en çok zaman alan işlem bulunur. Buna temel işlem (basic operation, most time consuming) denir. Bu genelde n'e bağlı döngülerdir.
- Birden çok döngü varsa en çok işlem yapan döngüye bakılır. Örneğin bir yerde iç içe iki döngü, başka bir yerde tek döngü varsa yalnızca iç içe döngüye bakılır ($n^2 + n \to n^2$).

```
for (i = 0; i < n; i++)      /* iç içe iki döngü: n^2, buna odaklan */
    for (j = i; j < n; j++)
        ...

for (i = 0; i < n; i++)      /* tek döngü: n, ihmal edilir */
    ...
```

Sırasız Dizide Sıralı Arama (Sequential Search)

Hocanın tahtaya yazdığı kod (ekrandan, 01:07:20):

```
int sequentialSearch(int n, int a[], int x)
{
    i = 0;
    while ((i < n) && (a[i] != x))
        i++;
    if (i < n)
        return i;
    else
        return -1;
}
```

- Dizi sırasız olduğu için elemanlara sırayla bakmaktan başka çare yoktur.
- Döngünün iki koşulu vardır:
  - `i < n`: dizinin sonuna gelinip gelinmediğini kontrol eder. Eleman dizide yoksa döngüden bu koşulla çıkılır.
  - `a[i] != x`: aranan elemanın bulunup bulunmadığını kontrol eder.
- Döngüden çıkınca `i < n` ise eleman bulunmuştur ve indisi döndürülür. `i == n` ise bütün elemanlara bakılmış ama eleman bulunamamıştır. Bu durumda `-1` döndürülür.
- Temel işlem, `while` döngüsündeki karşılaştırmadır.

En Kötü, En İyi ve Ortalama Durum

1. **En kötü durum verimliliği (worst case efficiency):** eleman dizide yoktur. Döngüye n kez girilir. $C_{worst}(n) = O(n)$ ve bu bir üst sınırdır.
2. **En iyi durum verimliliği (best case efficiency):** aranan eleman ilk elemandır. $C_{best}(n) = O(1)$. Her zaman karşılaşılan bir durum olmadığı için üzerinde durulmaz.
3. **Ortalama durum verimliliği (average case efficiency):** rastgele bir giriş için beklenen adım sayısıdır.

- Ortalama durum gerçek dünyada en sık karşılaşılan durumdur. Bu yüzden bazı sistemler maliyet nedeniyle ortalama duruma göre tasarlanır.
- Hocanın örnekleri: deprem anında herkes aynı anda telefona sarılınca hatlar kilitlenir. Üniversite sınavı sonuçları açıklanınca herkes aynı anda bağlanmaya çalışır ve sistem çöker. Bu sistemler, aynı anda o kadar çok kişinin hizmet istemeyeceği varsayımıyla daha ucuza kurulmuştur.

Sıralı arama için ortalama durum hesabı (ekrandan, 01:22:35):

- Aramanın başarılı olma olasılığı p'dir ve $0 \leq p \leq 1$.
- İlk eşleşmenin i'inci konumda olma olasılığı $\frac{p}{n}$'dir. Bu olasılık bütün konumlar için aynıdır.
- Bu durumda yapılan karşılaştırma sayısı i'dir.
- Başarısız aramada n karşılaştırma yapılır. Başarısız arama olasılığı $1 - p$'dir.

$$C_{avg}(n) = \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n}\right] + n \cdot (1 - p)$$

$$= \frac{p}{n}\left[1 + 2 + \cdots + n\right] + n(1 - p) = \frac{p}{n} \cdot \frac{n(n+1)}{2} + n(1 - p) = \frac{p(n+1)}{2} + n(1 - p)$$

- Köşeli parantez içi başarılı arama, ikinci terim başarısız aramadır.
- $p = 1$ ise (eleman kesin dizide): sonuç $\frac{n+1}{2}$ olur. İkinci terim sıfırlanır.
- $p = 0$ ise (eleman kesin dizide yok): sonuç n olur.
- Her iki durumda da katsayı ve sabit atılınca $C_{avg}(n) = O(n)$ bulunur.
- Ortalama durum hesabı çoğu algoritmada kolay değildir. Bu basit örnekte bile uğraş gerekti. Bu yüzden kitaplarda ve notlarda pek yapılmaz.
- İstisna hızlı sıralamadır: en kötü durumu $O(n^2)$'dir (örneğin dizi zaten sıralıysa). Ama ortalama durumu $O(n \log n)$ olduğu için en çok kullanılan sıralama algoritmasıdır. Bu yüzden onun ortalama durumu ayrıca incelenecek.

İkili Arama (Binary Search)

- İkili arama, verinin belli bir düzende olmasının işi hızlandırdığını gösteren bir algoritmadır. Dizinin sıralı olması gerekir.
- Mantık: ortadaki elemana bakılır.
  - Eşitse eleman bulunmuştur.
  - Aranan eleman ortadakinden büyükse, sol yarıdaki bütün elemanlardan da büyüktür. Sol yarı elenir.
  - Küçükse sağ yarı elenir.
- Her adımda yarısı elendiği için bu bir azalt ve fethet (decrease and conquer) algoritmasıdır. Ayrıntısı algoritma analizi dersinde görülecek.

Birinci örnek, $x = 21$ (ekrandan, 01:31:35):

İndis	0	1	2	3	4	5	6	7
Değer	2	7	9	12	17	21	23	37

baş	son	orta	Karşılaştırma
0	7	3	$21 > 12$, sol yarı elenir, baş = 4

baş	son	orta	Karşılaştırma
4	7	5	`a[5] = 21`, eleman bulundu

İkinci örnek, $x = 13$, eleman dizide yok (ekrandan, 01:51:00):

İndis	0	1	2	3	4	5	6	7
Değer	7	11	22	35	41	56	71	82

baş	son	orta	Karşılaştırma
0	7	3	$13 < 35$, sağ taraf elenir, son = orta − 1 = 2
0	2	1	$13 > 11$, sol taraf elenir, baş = orta + 1 = 2
2	2	2	$13 < 22$, son = orta − 1 = 1
2	1	–	baş > son, durulur. Eleman yok.

- Baş soldan sağa doğru ilerler, son sağdan sola doğru gelir.
- Başın sondan büyük olması, bakılması gereken bütün bölgelerin işlendiği anlamına gelir. Artık bakılacak eleman kalmamıştır.
- Her zaman olduğu gibi işlemin bitme durumunu dikkatli belirlemek gerekir. Eleman dizide olmayabilir. Olmadığı anlaşıldığı anda iş kesilmelidir.
- Derste bir soru üzerine hoca hatırlattı: önce `a[mid] == x` kontrolü yapılır. Büyük/küçük kontrolü bundan sonra gelir. Tahtadaki örneklerde yalnızca eşit olmama durumları gösterilmişti.

Akış diyagramı (ekrandan, 01:59:55):

```
start
  -> oku: n, a[n], x
  -> l = 0, r = n - 1
  -> found = 0
  -> (l <= r) && (!found) ?
       T: m = (l + r) / 2
          a[m] == x ?
            T: found = 1
            F: x < a[m] ?
                 T: r = m - 1
                 F: l = m + 1
          -> döngü koşuluna geri dön
       F: found ?
            T: yaz: m
            F: yaz: "yok"
          -> stop
```

- Derste bir öğrenci, ilk `if` kutusunun `True` kolunun bağlanmadığını fark etti. Hoca bağlantıyı ekledi.
- Öğrenciler kodun akıştan yazılabileceğini söylediği için kod derste yazılmadı.

İkili Aramanın Karmaşıklığı: Geriye Doğru Yerine Koyma

- En kötü durum: eleman dizide yoktur ya da en son adımda bulunur.
- Her adımda bir karşılaştırma (`a[m] == x`) yapılır. Sonra yarısı elenir ve aynı iş $n/2$ eleman için tekrarlanır.

Ekrandaki türetme (ekrandan, 02:12:10):

$$C_w(n) = C_w(n/2) + 1 \quad (n > 1), \qquad C_w(1) = 1$$

$$C_w(n) = C_w(n/2) + 1 = C_w(n/4) + 2 = C_w(n/8) + 3 = \cdots = C_w(n/2^i) + i$$

- Buradaki $+1$, her adımdaki karşılaştırmadır.
- Örüntü açıktır: i'inci adımda bölen 2^i olur.
- Yineleme $n/2^i = 1$ olunca durur. Çünkü tek eleman varsa bir karşılaştırma yapılır ve iş biter.
- $n/2^i = 1$ ise $n = 2^i$, yani $i = \log_2 n$ olur.

$$C_w(n) = C_w(1) + \log_2 n = \log_2 n + 1 \Rightarrow C_w(n) = O(\log_2 n)$$

- Bu yönteme geriye doğru yerine koyma (backward substitution) denir. Algoritma analizi dersinde ayrıntılı görülecek.
- $2^{10} = 1024$ elemanlı dizide sıralı arama en kötü durumda 1024 karşılaştırma yapar. İkili arama ise en fazla 10 karşılaştırmada sonuca ulaşır.

Soru: 0'ların Bitip 1'lerin Başladığı Yer

- Soru: Önce 0'lar, sonra 1'ler gelen sıralı bir dizide 0'ların bitip 1'lerin başladığı yeri $O(\log n)$ karmaşıklıkta bulun. Örnek dizi: `0 0 0 1 1 1 1 1 1 1`.
- Düz ikili arama yetmez. Ortada 1 bulmak tek başına bir şey ifade etmez.
- Derste öğrencilerle birlikte varılan mantık (ekrandan, 02:20:50):
  - Ortadaki eleman 1 ise sola doğru bakılır.
  - Ortadaki eleman 0 ise sağa doğru bakılır.
  - Durma koşulu: bakılan eleman 1 ve solundaki 0 ise ya da bakılan eleman 0 ve sağındaki 1 ise geçiş bulunmuştur.
  - Yani `a[m]` 1 ise solundaki elemanın, 0 ise sağındaki elemanın durumu önemlidir.
- Algoritma ilk 1'in indisini ya da son 0'ın indisini döndürebilir. İkisi de kabul edilir.
- Hoca akış diyagramını çizmedi. Öğrencilerin kendilerinin çizip çözmesini istedi.

Ders sonrası notlar

- $n \log n + n$**:** derste bir yerde ($n \log n + n$ ifadesi için) sonucun n olarak gösterilebileceği söylendi ve ekrana `n*lgn+n -> n` yazıldı. Doğrusu $n \log n + n = O(n \log n)$'dir. En anlamlı terim $n \log n$'dir. Hoca dersin başka bir yerinde bunu doğru söyledi: "$n \log n + n$ çıktıysa $n \log n$ diyeceksiniz". $n + \log n$ için ise sonuç $O(n)$'dir.
- B **değeri:** ekranda $n = 10000$ için $B > 10^{10}$ yazıldı. Doğrusu $B = 10000^2 + 10000 + 1 = 100\,010\,001 \approx 10^8$'dir. Sonuç değişmez: n büyüdükçe B, A'yı ($1\,000\,001$) çok geçer.
- **Faktöriyel zaman:** derste bir öğrencinin sorusuna "faktöriyel zamanda çalışan algoritma diye bir şey yok" cevabı verildi. Bu doğru değildir. $O(n!)$ karmaşıklıklı algoritmalar vardır. Örneğin bir dizinin bütün permütasyonlarını tek tek deneyen kaba kuvvet (brute force) çözümler, gezgin satıcı probleminin kaba kuvvet çözümü gibi. $n!$, 2^n'den de hızlı büyür.
- **Üstel zaman ve NP:** derste üstel zamanlı algoritmalar "nondeterministik polinomsal (NP) algoritmalar" olarak anıldı. Bu iki kavram aynı şey değildir. NP, çözümü polinom zamanda doğrulanabilen karar problemleri sınıfıdır. NP problemlerinin bilinen çözümleri çoğu zaman üstel zamanlıdır ama "üstel zaman" ile "NP" eş anlamlı değildir. Ayrıntısı algoritma analizi dersinde görülecek.
- **İkili aramada `left` güncellemesi:** altyazıda "left'i mid-1 yapacaksınız" gibi duyuluyor. Ekrandaki akış diyagramında doğrusu yazılıdır: `x < a[m]` değilse `l = m + 1`.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.